Permutations

AlgoExpert · Medium · Recursion

The problem

- All orderings of a list of distinct ints (empty input yields an empty list).

Intuition

- Build permutations by swapping each remaining element into the current position.

Approach

- 1. Recurse on index i
- 2. Swap each later element into position i
- 3. Recurse, then swap back

Java solution

```java
public class Permutations {
    public static List<List<Integer>> getPermutations(List<Integer> array) {
        List<List<Integer>> out = new ArrayList<>();
        if (!array.isEmpty()) permute(0, new ArrayList<>(array), out);
        return out;
    }

    private static void permute(int i, List<Integer> a, List<List<Integer>> out) {
        if (i == a.size() - 1) { out.add(new ArrayList<>(a)); return; }
        for (int j = i; j < a.size(); j++) { Collections.swap(a, i, j); permute(i + 1, a, out); Collections.swap(a, i, j); }
    }
}
```

Complexity

- Time: O(n * n!)
- Space: O(n * n!)

Recap

- Build permutations by swapping each remaining element into the current position.
- Complexity: O(n * n!)